Introduction of TOC

* **Automata.**

→ Study of abstract computing devices or machines.

* **Symbol:-**

→ An abstract entity.

eg:- a, b, 0, 1 .....

* **Alphabet:-**

→ Finite set / collection of symbols.

eg. $\Sigma = \{a, b, c\}$

$\Sigma = \{0, 1\}$

* **String:-**

→ Sequence of symbol:-

eg. $\Sigma = \{a, b\}$

strings = {a, b, aa, ab, ba, bb}

bb → length = 2

→ If the length of string is zero, then, it is known as Empty / Null string.

→ It denoted by $\in$ or λ.

(If length is given - finite)

* Language :-

→ Collection of strings.
→ Where strings are restricted over given alphabet.

eg: $\Sigma = \{a, b\}$

L_1 = Set of all strings length of 2.

$\{aa, ab, ba, bb\}$

Where language L_1 is finite.

L_2 = Set of all strings which start with a,

$\{a, ab, aab, aaa \ldots\}$ → infinite.

* Length of string :-

→ no. of symbols in a string.

Ex: $|\{a, b\}| = 2$

$|\{\Sigma\}| / |\Sigma| = 0$

* Prefix of a string :-

→ If $\omega = \{x, y\}$ for some string y then x is a prefix of ω.

Ex: $w = \{0011\}$

$\{0, 011\}$, $\{00, 11\}$ $\{001, 1\}$.
x y x y x y

* Suffix of String:-

if $w = xy$ for a string x, then y is a suffix of w.

Ex:- {0011}

001 ✓, 00 11 ✓, 0 011 ✓

* Kleene closure :-

→ It is denoted by * (Asterisk) after the name of Alphabet.

i.e. Σ^*.

→ This notation is also known as Kleene Star

Eg: $\Sigma = \{a, b\}$

Σ^0 = Set of all strings of length '0' = $\{\varepsilon\}$.

Σ^1 = Set of all string of length '1' = $\{a, b\}$

$\Sigma^2 = \Sigma^1 \cdot \Sigma^1 = \{a, b\}\{b, a\} = \{aa, ab, ba, bb\}$ → len 2.

$\Sigma^3 = |\Sigma|^3 = 8$ = Set of all string length 3.

$\Sigma^n = n = 2^n$ ✓

$\Sigma^* = \Sigma^0 \cup \Sigma^1 \cup \Sigma^1 \cup \dots$

$\Sigma^* = \Sigma^+ \cup \{\varepsilon\}$ [Set of all string of all length over $\{a, b\}$]

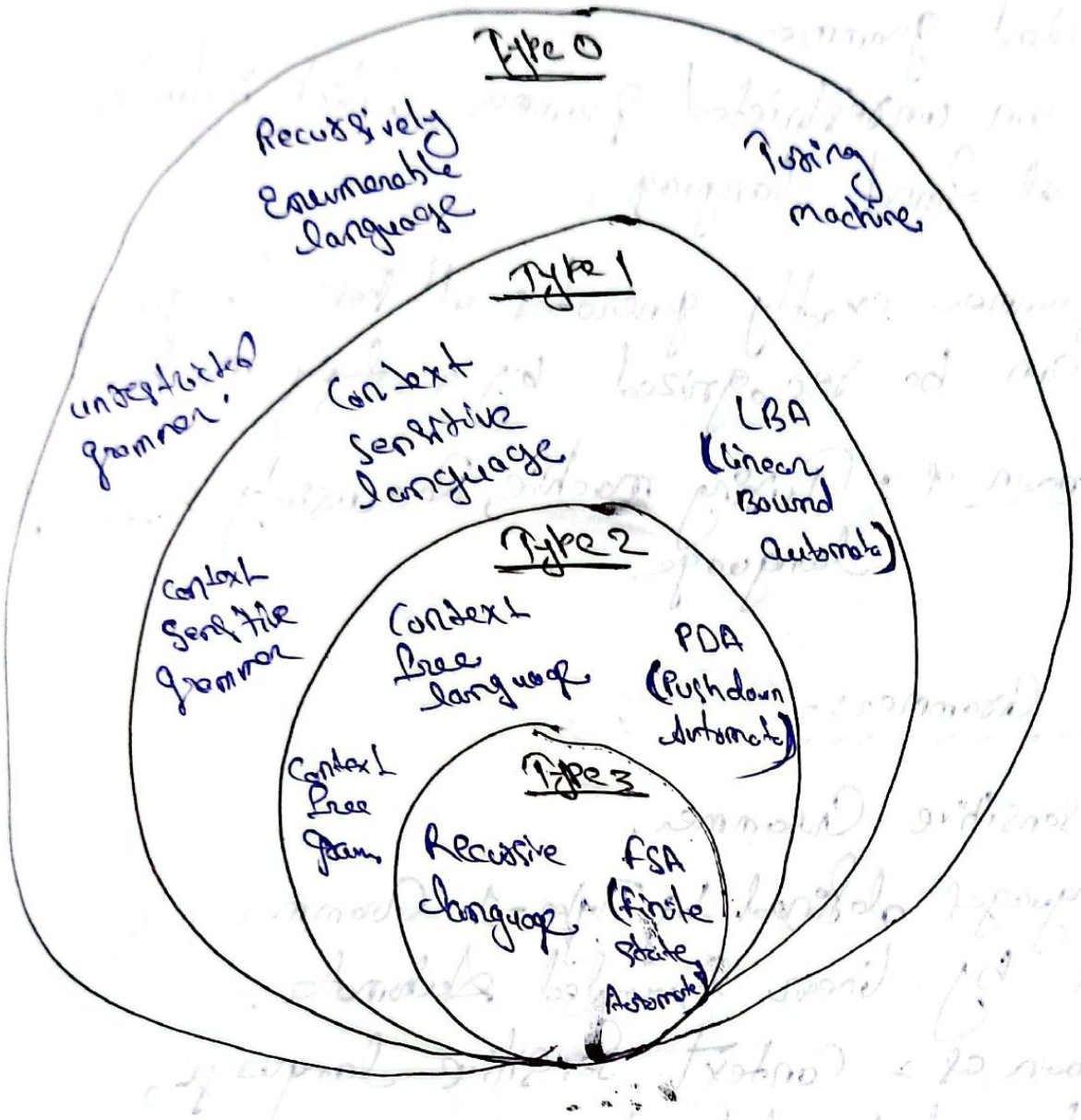
* Grammer:

→ It enumerates string of language.
→ It is a finite set of rules for defining a language
→ A grammer is ~~Set~~ defined as 4 tuples. (VTPS)

V = Set of non-terminals
T(Σ) = Set of input terminals.
P = finite set of production rules.
S = Start of Symbol.

(Getting a string from a grammer = Derivation)

Chomsky Hierarchy:-



It consists of 4 classes :-

Type 0

Type 1

Type 2

Type 3

(A) Type-0 Grammer :-

→ Unrestricted grammer.
→ This is an unrestricted grammer, which includes almost all formal language.
→ This grammer exactly generates all those languages which can be recognized by a turing machine.
→ Also known as : Turing machine, Recursively Enumerable language.

(B) Type-1 Grammer :-

→ Context sensitive Grammer.
→ The languages defined by Type-1 Grammer are accepted by linear Bounded Automata.
→ Also known as : Context sensitive language, linear bounded Automata

(C) Type-2 Grammer :-

→ Context free grammer.
→ Accepted by PDA.
→ Context free language, Push Down Automata

(D) Type-3 Grammer :-

→ Regular grammer.
→ accepted by finite state Automata
→ Regular language, finite Automata

* Finite Automata :-

→ machine with fixed amount of computational memory, accepts Regular languages.

→ Applications :- Useful for modeling clocks, communication protocols, Vending machine, some control system etc.

* PDA :-

→ Finite Automata with unbounded structure memory in the form of a push down stack.

→ Applications ⇒ Useful for modeling parsing, compilers, Programming language design.

* Turing machine :-

→ Finite Automata with unbounded tape accepts or enumerable recursively language.

⇒ Equivalent to RAM & various programming language model.

→ Applications :- model for general purpose computing.

* Regular expressions & Finite Automata :-

* Finite Automata :-

FA without output

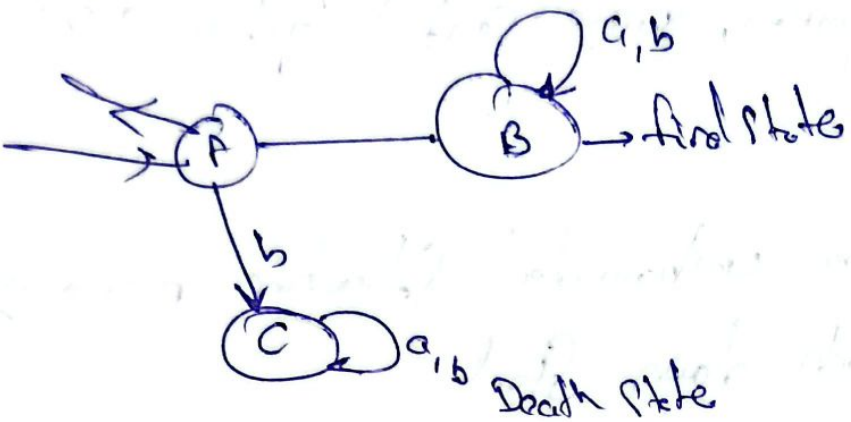
DFA
NFA
ε-NFA

~~DFA~~ with output

① moore machine
② mealy machine

* DFA:-

→ A finite Automata is defined as like tuple.



Q = Set of all states (q_0, q_1, q_2)
Σ = Inputs (a, b)
q_0 = Initial state (A)
f = final state (B)
δ = Transition fn.

DFA	NFA
1) Each DFA can be an NFA	Each NFA can't be a DFA.
2) DFA & NFA same power, DFA can be translated to a NFA	NFA can't.
3) Both can have multiple final states	—
4) Compiler use DFA in lexical analysis.	Compiler doesn't use NFA
5) Backtracking is possible	Not always possible.
6) Require more space	Require less space.

ε-NFA :-

→ In NFA, moving on Null (ε) is allowed. i.e. it can move without reading any symbol.
→ used to identify regular languages
→ It makes NFA more powerful, as it can transit without any ~~output~~ input.

* Moore v/s Mealy machine :-

(i) Basic Definition :-

→ The output of moore machine depends only on the current state, where as the output of a mealy machine depends on both the current state and the input.
→ In moore machine, the output table is associated with state, while in a mealy machine, the output table is associated with input.

(ii) Structure / Components :-

- Common :
  State (Q)
  Initial state (q_0)
  Output Alphabet (Λ)
  Input Alphabet (Σ)
  Transition function (δ)

Differences

output fn (λ) = $Q \rightarrow \Lambda$ (moore)
output (λ) = $Q \times \Sigma \rightarrow \Lambda$ (mealy)

(3) Example:

- Moore machine, Current State = q_1 then the output = 1 no matter what the input is.

State	output
q_0	0
q_1	1
q_2	0

- Mealy machine the current State is q_1 and if input = 0 then output = 1. And if the current State = q_1 & input = 1, then output 0.

Transition	output
$q_1, 0$	1
$q_1, 1$	0

(4) Speed of output :-

- Moore machine, output received after a State change, So it is slow
- Mealy machine, output received immediately with the input, fast

(5) Number of States :-

- Moore machines have more States because the output is linked to States, whereas mealy machines have fewer States because the output is linked to transitions.

6) Conversion :-

→ more to mealy conversion is possible and leads in lesser states. mealy to moore conversion is also possible, but its results in more states.

* Regular Expression (RE) :-

→ method used to define regular languages.
→ uses symbols and operators to define patterns of strings.

for example :-

a|b = a or b (Union in Regular language)

a* = Zero or more occurance of a.

→ Precedence (Highest to lowest)

(1) kleene Star (*)
(2) Concatenation
(3) Union (|)

* Turing machine :-

→ mathematical model of computation that can simulate any algorithm.
→ By Alan Turing (1936)
→ Theoretical computation machine, which shows that a computer can solve certain problems and not others.

* Structure of Turing machine :-

(A) Tape :-

→ An infinite strip on which input is given.
→ During computation, the machine can read/write/erase on it.

(B) Tape head :-

→ A pointer that is on the current cell.
→ It can read/write and move (left/right)

(C) Finite control / state :-

→ A finite set of states ($q_0, q_1, q_2 \ldots$) shows the current status of the machine.

(D) Transition function :-

→ It decides what action will be taken on which input symbol. Denoted δ (delta).

* ID :-

→ Instantaneous Description.
→ An ID describes which state the machine is in, what is written on the tape, and at which position the head is.

* Language accepted by TM :-

→ A Turing machine accepts a language if, after processing the input string, it reached the accept state.

→ 2 mode of acceptance.
(a) Acceptance by final state.
(b) Acceptance by halting state.

→ TM accepts Recursively Enumerable (RE) or Type 0 lang.

* Variant of TM :-

(i) multi tape (multiple tapes but equivalent to single tape TM)

(ii) multi track (multiple tracks on a single tape (parallel))

(iii) Non Deterministic (multiple tracks possible for one input)

(iv) Two way infinite Tape TM. (Infinite tape on both sides)

* All of these have Computational Power equivalent to the Standard Turing machine.

* Universal Turing machine :-

→ A TM that can simulate the behaviour of any other TM.
→ It is given both input and program (type of TM)
→ It forms the basis of the concept of a general-purpose computer.

* Church's Thesis :-

• "Whatever is computable by an effective method is computable by TM".

Features	finite Automata	PDA.
(1) memory	No external memory.	Stack.
(2) Power	less Powerful (Regular lang.)	more powerful. (context free)
(3) Example.	String ending with "ab"	Balanced parenthesis & Palindromes.

* Working of PDA :-

→ When a PDA reads any input, it also pushes or pops symbols onto/from the stack.

→ A PDA can accept a string in two ways.

(a) By final State :-
When the input finishes and the machine reaches the final state.

(b) By empty Stack :-
When the input finishes and the stack becomes empty.

→ Both these method are equivalent, meaning by using either of them, the same languages can be recognized.

* DPDA (Deterministic PDA)
→ Only one possibility each step.
* NPDA (Non-deterministic PDA)
→ more than one possibility at each step.
* NPDA more powerful (It accepts more lang.)
* DPDA accept a few languages.